FREE PREVIEW

MX: THE INTRO

Chapter zero of the guide to building a web that works for AI agents — and everyone else

CONTINUE WITH

MX: THE HANDBOOK

Everything you need to design the web for AI agents — and everyone else

MX: THE PROTOCOLS

The formal standards and specifications for an AI-readable web

This is not the full system.
It's the starting point.

Unlock the full system →

mx.allabout.network

More AI traffic

More conversions

More revenue

Future-proof your business

Published and printed by mxprintworks.com, 120 Main Street, Largs, Scotland.

© CogNovaMX. First published 2026. All rights reserved. For information about permission to reproduce selections from this book, write to Permissions, MXPRINTWORKS, LPC Design + Print, 120 Main Street, Largs, Ayrshire, KA30 8JN Scotland UK.

First Edition. Printed in the United Kingdom of Great Britain and Northern Ireland. Cover Design by Scott McGregor. Book Design by Tom Cranstoun.

The prose in this book uses neutral English. Code names (HTML elements, CSS properties, Schema.org types) keep the spelling their defining standards use, in every language: they are code, not prose.

For information on booking the author for an event please visit mx.allabout.network/books/the-author.html.

Contents

MX: The Introduction 4

Purchase the Books 28

The AI Tipping Point - CMS Kickoff 2024 29

Work with the Author 38

MX: The Introduction

Understanding the machines reading your website, and the revenue at stake.

The tool you have to know too much about

You are part-way through a memo, and you stop, not to write but to choose. The newest, biggest model or the cheaper one? The mode that thinks, or the one that answers straight away? You pick the biggest, newest option you can, because you have no way to know what the smaller one would have given up. A colleague runs the same week of work twice and watches the bill climb from a few dollars to more than six hundred, the same tasks, a newer model underneath, and no explanation attached to either run.

It is an oddly unproductive moment in a powerful technology. We spend our attention on the machinery, on model names and modes, instead of on the work. The task never tells us which model it needs, because the file it lives in says nothing about itself: not what it is, not what it is for, not who is accountable for it. So the burden falls on the person, every time.

This short book is about moving that burden off the person and into the file. That is what Machine Experience does, and why the revenue in the pages that follow turns on it.

Revenue you cannot see leaving

Machines are mediating commerce right now. They compare your products, check your availability, and recommend you or your competitors. The commercial question is straightforward: are they completing transactions on your site, or moving on to someone who made it easier?

Adobe's Holiday 2025 data makes the scale visible. AI referrals to retail sites surged by 700%. Travel referrals rose by 500%. Conversion rates from AI-referred visitors now lead human traffic by 30%, and those visitors are 33% less likely to bounce. Machine-mediated commerce moved from experimental to revenue driver in a single quarter.

In January 2026, four major platforms launched machine commerce systems within eight days: Amazon Alexa+ (5 January), Microsoft Copilot Checkout (8 January), Google Universal Commerce Protocol (11 January), and Anthropic Claude Cowork (12 January). What industry analysts predicted would take 12-24 months compressed to six months. Machine-mediated commerce is infrastructure, not experiment. By June 2026 the payment rail itself had moved inside the chat window: Visa embedded its network in ChatGPT, and an agent now arrives at any merchant that takes the card, with no integration project on your side. The agent can pay anywhere; it can only buy what it can read.

The machines want to buy from you. The question is whether your content gives them what they need.

The discipline that produces all of this, what you write, what you film, what you record, what you scan into a PDF, what you table up as a dataset, the contracts your legal team drafts, the briefs your agency turns around, is Content Ops. Creating it, managing it, improving it, publishing it, distributing it, archiving it, retiring it. The web is one channel. There are dozens. MX is what makes the work of Content Ops survive every channel, and especially the moment when an AI agent, or any other system, encounters one of those files outside the environment that produced it. Without that survival layer, the work was authored once and then quietly forgotten by every machine that touches it next.

Your website is a fraction of your content estate. Contracts, policy documents, product specifications, and technical reports do not live on the web - but AI agents inside enterprise tools are already reading them, inferring what they can, guessing the rest. Being on the web and being machine-readable are not the same thing. A document on a public server is still opaque to an agent that has no context about what it is, who wrote it, what version it represents, or what claims it makes. Web accessibility solves discoverability. MX solves comprehension.

The invisible users

There is a new class of user on your website, and they are called invisible for two reasons. They are invisible to you: they blend into analytics, arrive once, and leave no trace. The interface is also invisible to them: they cannot see animations, styling, toast notifications, loading spinners, or any of the visual cues your site uses to communicate. These are machines visiting your website and performing actions without your awareness.

Most companies do not track AI bot traffic. Some block it entirely through robots.txt directives or Cloudflare identity checks, while simultaneously wondering why their AI-referred revenue is lower than competitors'. Modern AI browsers do identify themselves in their User-Agent strings, but those strings are trivially spoofed by any developer. You cannot know which type of machine is visiting, or how capable it is.

The patterns MX requires for these invisible users (explicit structure, semantic HTML, persistent state) also happen to benefit users with disabilities who rely on screen readers and keyboard navigation. That convergence is real. The business case for action is what drives the investment: machines are mediating commerce, answering health questions, comparing universities, and completing government transactions on behalf of humans. When your site fails them, you lose that business silently.

The convergence extends to the files you publish, not just the pages. A typical company ships PDFs alongside its web content: white papers, product datasheets, regulatory filings, contracts, training materials. Most of those PDFs are byte streams without structure tags or alt text, which makes them unreadable to screen-reader users (one in four people in the European Union has some form of disability) and also poorly extractable by machines - AI agents, search-engine crawlers, and every automated system that reads documents. The European Accessibility Act, in force from 28 June 2025, applies financial penalties of approximately ten thousand euros for minor violations and one hundred thousand euros or

more for major or repeated non-compliance, with significantly higher figures available against large enterprises. Producing tagged, declared, verifiable PDFs is the same work as producing accessible web pages: tag the structure, declare the conformance, record the check. MX makes the resulting documents accessible as a side effect of making them machine-readable, exactly as it does for HTML.

This is also where UNESCO's fairness principle lands. The Recommendation requires AI systems to respect fairness, non-discrimination, and digital literacy. MX's WCAG requirement and its insistence on existing primitives - Schema.org, semantic HTML, ISO formats, BCP-47 language tags - are the technical expression of that requirement. All users, regardless of disability, language, or device, deserve content that works for them. Accessibility is the visible edge of that commitment; machine-readability is the other edge.

Three failures the web cannot fix itself

Three interlocking structural failures block machine access, and no amount of AI improvement can resolve them.

Hostile design. Every website demands something before delivering value: accept cookies, dismiss the banner, close the pop-up, navigate the carousel. A toast notification that vanishes after three seconds is invisible to software reading the DOM on a half-second cycle. Tabs hide content the machine never requested. Accordions and show-more toggles collapse it before the machine arrives. The hostile web is architecturally incompatible with machine access, and machines will not get better at reading it. The ambiguity in your markup is permanent until you remove it.

Content imprisoned in platforms. Every CMS traps content in its own format, its own database, its own API. Machines cannot read any of it, because the content was designed for human eyes inside a proprietary system, not for machines operating across open standards.

Structured data is not enough. Schema.org describes. MX executes. This distinction is critical: metadata is not just documentation of a fact. Each metadata field is an active instruction that tells a machine what to do next - what to verify, what to check, when to stop trusting. Schema.org JSON-LD addresses what machines can *read*, but not what they can *do*. It does not fix hostile design. It does not free content from lock-in. It does not govern who may access the content, under what terms, for what purposes. Structured data is one layer in a practice that must span the entire interaction surface.

MX is not Generative Engine Optimization. GEO - the vendor category built around tuning content so LLMs cite it - asks one question: how do I get an AI to mention my page? MX asks a different question: can any machine, in any format, at any time, find any document in a corpus, verify it is genuine, and know whether it is current? GEO tunes a single pathway. MX builds coverage across every pathway, because which pathway any given machine will take is unknowable in advance. Implement MX and GEO improvement follows automatically. The reverse is not true.

There is a mechanical reason GEO spend leaks away. Most agents read a page the way `curl` does, taking the raw HTML the server sends and never running the JavaScript; only an agent driving a full browser sees the rendered page, and that path is the costly exception, not the rule. Anything that appears only after the page renders is invisible to the machines that matter, because it was never in the HTML they read. MX puts the meaning in the raw response, where the fetch lands. And it asks something back: a machine has to learn to read MX for the metadata to count. Publishing structure is one half of the bargain; building agents that use it is the other. Asking a page for Markdown instead of HTML strips the same value from the other side: the conversion keeps the prose and discards the structured data and embedded metadata, so the machine throws away the very signals it came for.

There is a sharper failure than leaked spend. The tactics sold as GEO - publishing at volume, stamping out templated pages, marking up every claim - can do more than fail to help; they can get a site demoted or fully de-indexed in ordinary search, quietly, with no warning and no penalty notice, even as its AI citations climb for a while. And because answer engines retrieve from a search index, those citations collapse a cycle behind the rankings: you lose both, because they were always the same asset. Traditional SEO - a crawlable site, semantic HTML, content a person actually wanted - is the backbone every citation stands on, and there is no "just do GEO" that skips it. Tune a page because it is worth reading, never for its own sake.

The same gap shows up wherever publishers chase ranking rewards through markup alone. SEO, GEO and AEO each tell a content team to decorate the page; none of them validates the page. Ranking systems have already responded. FAQ and Q&A markup, used most aggressively to game rich-result slots, are now deprecated as rich results. Schema.org keeps expanding on one side; Google keeps withdrawing rewards on the other. The contradiction is structural: schema markup describes what a page claims to be, but cannot tell a machine who made the assertion, when, or whether to trust it. That is the provenance gap. It widens every time an agent acts on structured data without a way to verify it, and the incentive to game the markup grows with the weight placed on it. MX closes the gap by carrying provenance with the document rather than leaving it to inference.

There is a second gap right next to provenance. SEO, GEO, AEO and JSON-LD all answer the same question: how does a machine *find* this page. Once the machine has it, the page is silent about what to do with it. None of them carries when the page was published, when it expires, who maintains it now (distinct from who wrote it), what its canonical URI is, or whether a newer version has replaced it. MX does. The MX field dictionary names those six signals as first-class top-level fields every cog carries: `created` for publication date, `expires` for end of validity, `originator` (surfaced as `author`) for the immutable creator, `stewardship.steward` (surfaced as `maintainer`) for the mutable accountable contact, `canonicalUri` for the durable address, and `status` plus `supersedes` / `supersededBy` / `replacedBy` for the supersession chain. This is the lifecycle gap. The discovery layers solve *find me*; MX solves *now what*.

Not every document needs every field. MX groups them into profiles: a short, named list of what one kind of document must carry, so a blog post, a contact record and a risk register entry are each checked against their own fields. A document that a machine can run adds a security profile, saying what is at stake and which check stands in the way before anything

executes. *MX: The Protocols* gives profiles a chapter of their own.

One caution runs underneath all of this: do not overdo it. The temptation, once you can mark anything, is to mark everything, and an agent told to "add structured data" will do exactly that. Decorate every claim and you teach machines to ignore the markup, the same way gamed FAQ schema lost its reward. Mark what is material, the claims that change a decision, and leave the rest as plain prose. Knowing which is which, and who owns the data model that decides it, is human judgement, not an automatic step. This introduction only names the discipline. *MX: The Handbook* sets out the principle, and *MX: The Protocols* develops it in full. If this is the part you need, the next step is to buy the books.

The diversity matters in practice. There is no single "LLM" to tune for: ChatGPT, Claude, and Gemini have different ingestion paths, different fetch patterns, and different trust signals. A tactic that lifts your citations in one can do nothing in another, and the gap widens every time a vendor tightens its rules, changes its system prompt, or ships a new model. New readers arrive every quarter - agents you have never heard of, parsing your pages by rules you cannot predict. The plumbing is what carries you across that churn: right MIME types, a discoverable sitemap, an llms.txt that is actually wired up, structured data that matches what is on the page. Four items, each a configuration change in tools you already use. Tactics will reshuffle every few months. Plumbing does not.

The confusion runs deeper than most buyers know. A brand name is not one machine. "Claude" or "ChatGPT" is a label over a web chat, an editor plugin, a phone app, and an assistant embedded in some other product, each running a different model under different rules, each behaving differently. The steady voice reads as one personality, and there is no personality there at all. Two of them under the same brand do not share a memory: what you told one, the next has never heard. The only thing that carries across all of them is what you write down where every one of them can read it.

Machine Experience (MX) addresses all three - and it is not an internet problem. It is a document problem. Contracts, manufacturing specifications, government archives, medical protocols, scientific papers: the substrate of every enterprise. The same structural failures that block machines on a retail website block robots reading maintenance manuals, autonomous vehicles parsing safety procedures, and industrial systems consuming manufacturing specifications. MX covers the entire asset space. Every document any machine might encounter - in any format, on or off the internet - is within scope. The document is the problem. The document is also the fix.

MX sits alongside UX, SEO, and accessibility as a fourth discipline in web practice, and it shares the same foundational principle Steve Krug applied to human usability: don't make me think. When a machine has to think (inferring meaning, guessing formats, imagining context that was never written down) you have failed at MX. The difference is that a frustrated human visitor might complain, come back, or call your support line. A machine says nothing. It routes to a competitor whose HTML is easier to parse, and you never know it happened.

One principle underlies the practice: design for the most constrained machine and every machine benefits. You cannot know which system is consuming your document - user-agent strings are easily spoofed, and the range of machines reading documents now extends far beyond web browsers and AI assistants to robots, industrial systems, and devices that have no concept of a visual interface at all. Design for the worst and every system gains.

The common objection is that AI will get better and eventually handle poor markup. The precise version of that objection fails in a specific way: hallucination is an infrastructure problem, not a model problem. Better models hallucinate with more confidence on ambiguous data, not less. Foundation models will improve. The trend also runs the other way: small language models are coming to every phone, industrial controllers are consuming documents in real time, and the class of machines that need to read your content is expanding faster than any single model can accommodate. The most constrained machine is not a historical edge case; it is the near-term majority. Designing for it is not a hedge; it is preparation for what is already arriving.

Four commitments follow from that principle:

- **Typed over prose.** Prices, dates, identities in typed fields with agreed formats (ISO 4217, ISO 8601, BCP-47). Prose alongside, for humans.
- **Bound, not inferred.** A number on a page belongs to the entity it describes. Structural binding stops the machine guessing which entity a value applies to.
- **Human always in the loop.** Outputs remain readable and reviewable by both. Anything legible to only one reader (human or machine) is a regression. UNESCO's seventh principle - human oversight and determination - says the same thing in policy language: if no human can review what a machine did, the principle has already failed.
- **Build on what is already known.** Schema.org, JSON-LD, microdata, semantic HTML. Existing primitives, used with discipline.

The contract your CMS already made

Here is the part most vendors have not caught up with. A content management system is not just software. It is the controller of a publishing point, and the moment you publish through it, it takes on an implicit contract with you: we will make your content visible to your audience in a way that benefits you. Every CMS sells on that promise. Be found. Reach your people.

Nobody reopened that contract, but the world underneath it changed. "Your audience" used to mean humans with eyes. It now includes machines as well, the agents comparing your products and the assistants answering questions about you, whether anyone wrote that into the brochure or not. The promise did not break; it simply stopped being honoured. The CMS still controls the output layer, so the duty to make your content legible to that new audience sits with the vendor, not with you. You delegated the templates the day you bought in, and you cannot fix them on your own.

And the failure stays invisible until it is not. You read your own site in a browser, where it looks fine, so you keep trusting the platform that serves it, right up to the day an agent cannot complete a purchase, a competitor gets cited and you do not, or an audit puts the gap on a page. That is the moment trust in the vendor collapses, all at once rather than gradually. The Protocols develops this in full: a CMS is a controller of a publishing point, and the vendors who fail to update this contract will lose the trust they were built on.

When it goes wrong

A buyer decides to purchase a Kindle Scribe Colorsoft from Amazon at £570, credit card ready. The UK listing says "coming soon." His AI assistant can find the product and quote the price, but cannot complete the purchase, check stock across regions, or tell him when to come back. The machine could not finish the transaction. The revenue went nowhere. Amazon made it surprisingly difficult to take his money.

In January 2026, two high-speed trains collided near Adamuz in Spain. Forty-six people died. Within hours, pricing algorithms at airlines and car hire firms detected surge demand and raised prices by up to 200%, while families scrambled to get home. The algorithms worked correctly. Demand up, supply down, price up. Nobody had given them the governance rules that would have flagged a national tragedy. Spain's national ombudsman launched an investigation. The reputational damage will outlast the revenue gained.

Both failures share one root cause: machines acting on incomplete data. Commerce loses revenue. Public services lose trust, and sometimes lives. The technical solution serves both.

> A machine that acts on incomplete data has not malfunctioned. It was never given the missing fact, or the rule for when to stop and ask. Reliability is not the model growing cleverer. It is the machine left with less to guess, and told to check before it acts.

Why this matters to you

Whether you sell products, inform patients, publish research, or guide citizens through government services, machines need explicit structure to complete those actions.

A machine cannot reliably extract your price because "€2.030,00" could be two thousand and thirty euros or two point zero three. It cannot find your stock status because it lives in a JavaScript variable the DOM never exposes. It cannot complete your checkout because the "Confirm" button is a `<div>` with no semantic meaning. It skips you in recommendations and you never see it leave.

These are standard patterns on millions of websites. Every one of them breaks for automated visitors.

Consider what that looks like in practice. A buyer asks their shopping agent to find noise-cancelling headphones under £200. Two competing sites appear in the agent's crawl. The first has prices behind "See pricing" buttons, specifications locked in image-based charts, and stock status behind a login gate. The agent reports back: "I found several options but couldn't verify current prices or availability." The second has prices in Schema.org `Offer` markup, specifications in structured lists, real-time stock in the `availability` property. The agent reports: "Product X at £179, 4.7 stars from 2,800 reviews, ships in two days. Shall I proceed?" The second site gets the sale. The first was never considered.

The failure is silent. When an agent cannot read your page, it does not send a complaint. It routes to a competitor whose HTML is easier to parse. No analytics signal. No error report. Quiet abandonment at the moment of intent, and you will not find it in your dashboards.

When an agent cannot read your page accurately, it does not stay silent - it confidently tells the user what it thinks it found. "Your AI said the price was £49." "ChatGPT told me you were open on Sundays." "The agent said you ship to Canada." Every ambiguity in your HTML becomes a support ticket, multiplied across every user who trusted the answer. Structured data is a customer service investment as much as a discovery one.

The scale is measurable. Most sites lack proper semantic HTML. The majority have no llms.txt. More than half have partial or missing Schema.org structured data. A significant proportion block major AI crawlers outright, while simultaneously wondering why their AI-referred traffic lags behind competitors'. The infrastructure the machine economy expects simply is not there.

Blocking is rarely the single decision it looks like. Some crawlers feed one company, so turning one away costs you only that one. The crawler behind Common Crawl feeds a shared archive much of the industry trains on, so the same one-line block can quietly drop you from nearly all of them at once. When a security plugin treats "block AI" as one switch, it makes that call on your behalf, and it does not tell you which bots carry which cost.

Check your own server logs today. Search your access logs for GPTBot, Claude-Web, PerplexityBot, and Bytespider. These visitors arrived, read your content, and either used it to answer someone's question about your business, or failed and moved on to a competitor. You have no visibility into which outcome occurred unless your content is structured for machines to read accurately. This diagnostic takes five minutes. The answer tells you whether MX is urgent or merely important.

> Structure cuts both ways. Machines read your pages, and increasingly they write them: research comparing 61,608 AI-generated stories against human writers found that document structure alone identifies the machine author 93% of the time (Russell et al., 2026). What you publish carries a structural signature, and both your readers and their machines can read it.

How machines actually access your site

There are two fundamentally different mechanisms of machine access, and they operate on completely different timescales.

Training-time ingestion: historical knowledge building

During model training, large language models ingest web content through datasets like Common Crawl, massive archives of crawled websites that form the knowledge base machines draw upon when answering queries. This ingestion happens months or years before machines interact with users, making it fundamentally historical rather than real-time.

Common Crawl operates as a web archive service that periodically crawls public websites, respecting robots.txt directives and sitemap.xml files. The crawled content becomes part of training datasets that teach models about the structure, patterns, and information available on the web. When machines reference your product specifications or company information without visiting your live site, they're drawing from this historical knowledge base acquired during training.

This training-time access has specific characteristics. It's indirect: your website doesn't interact with the model; it interacts with crawlers that build datasets used later for training. It's historical: the content the model learns from could be months or years old by the time users interact with the trained model. Crucially, it respects robots.txt: ethical crawlers honour your access control directives.

Inference-time access: real-time direct interaction

During user queries, machines may directly access your website in real time. When someone asks ChatGPT "What laptops does Example Store currently offer?", the system might fetch your live website using browser automation, API calls, or direct HTTP requests. This is inference-time access, software retrieving current information while processing a specific user query.

This direct access operates differently from training-time ingestion. It's direct: the machine fetches your website while the user waits. It's real-time: the content retrieved is your current live site, not a historical snapshot. It's specific: the system targets particular pages relevant to the query, rather than crawling your entire site. Critically, it may not respect robots.txt: software executing user queries might access content regardless of crawl directives, treating robots.txt as guidance rather than absolute constraint.

There is a further distinction most businesses miss entirely. There are two states of any web page: the served HTML (the raw markup delivered from the server, before any JavaScript has run) and the rendered HTML (what appears in a browser after the DOM has been fully manipulated). Most businesses test only the rendered state. Server-side machines, including

the crawlers that build training datasets and the agents that answer live queries, receive the served HTML. If your prices, stock status, and product specifications live in JavaScript variables, those machines see none of them.

This points to a deeper principle. Every file has two simultaneous audiences: the human who sees the visual surface, and every machine that reads the source. Those two readings of the same bytes are not the same experience. The structured data, governance metadata, and MX fields that travel in the source layer are invisible to human visitors. A `"@type": "Article"` declaration in the JSON-LD does not change what any human sees. It changes what every machine reading the source knows. The source layer is the machine-readable contract; the visual surface is for the human visitor. MX is the discipline of authoring both deliberately. The Protocols book develops this in full; what matters here is the starting point: if you only ever test and review the rendered surface, you have no view of what machines are reading.

The distinction matters for authentication and permissions. Content blocked by robots.txt won't enter training datasets through Common Crawl, but inference-time machines actively executing user queries might access that content anyway. If your content truly must remain private, authentication mechanisms (login requirements, API keys, IP restrictions) provide reliable access control while robots.txt alone does not.

Why both mechanisms need the same fix

The good news: the same MX patterns serve both mechanisms simultaneously.

Sitemap.xml serves both. Semantic HTML serves both. Schema.org structured data serves both particularly well; during training, JSON-LD blocks teach models about entity relationships and product attributes; at inference time, machines parse that same JSON-LD to extract current pricing and availability without interpreting prose.

There is one notable exception. The llms.txt file has a structural problem that limits its reach. It is served as a text or markdown MIME type, not HTML. Common Crawl ingests HTML pages; it does not typically ingest non-HTML files. Training-time crawlers may never discover it. At inference time, automated systems typically do not fetch it either; a system answering "What laptops does Example Store sell?" goes straight to product pages, not a site-level directory file. To close this gap, publish the same content as an HTML page and include it in your sitemap.

The Gathering (the independent standards body that governs MX specifications) addresses this in the MX Agent Directory Discovery note. The note is a draft standard that applies to any agent-directory file your site might publish, llms.txt today, ai.txt or whatever follows tomorrow. It defines three things any team can verify in an afternoon. First, transport: serve the file as `text/html` so the largest training crawlers can ingest it. Second, discovery: list the file in your `sitemap.xml`. Third, resilience: include a `<link>` tag in every page that points to the file, so the reference survives even when your page body is built by JavaScript and is

empty for the crawlers that do not run scripts. None of the three requires a new piece of infrastructure. Each is a configuration change in tools you already use. The note exists because none of the three is what most teams currently do, and the file is invisible until all three are in place.

Training vs learning vs codification

An important distinction: machines do not get "trained" when they visit your site. Their neural network weights are fixed. What actually happens involves three distinct layers:

Layer	What Happens	Cost	Persistence
Training (AI company's work)	Models learn general capabilities from datasets like Common Crawl: HTML syntax, Schema.org patterns, structured data extraction	$50-100M per model (one-time)	Permanent but historical
In-context learning (per session)	Machine reads your HTML, understands your specific patterns, navigates your site	$0.10-1.00 per session	**Disappears when session ends**
Codification (what MX creates)	Semantic HTML and Schema.org markup capture knowledge permanently in the page itself	Time to implement (one-time)	**Permanent and reusable**

Without MX, every machine must re-learn your site structure from scratch every session: re-read, re-discover navigation, extract information, then forget everything. With MX, the structure IS the captured knowledge. These systems spend zero time learning and 100% of their processing capacity extracting accurate information. Implement once, every visitor benefits forever, saving compute and energy costs as well.

That saving is not incidental. UNESCO's Recommendation on the Ethics of Artificial Intelligence names environment and ecosystem flourishing among its four core values. One of MX's founding principles carries the same instruction: choose what is better for the planet - reduce compute, reduce inference, reduce energy. A machine reading a structured cog instead of guessing at unstructured prose pays a smaller energy bill, and that bill multiplied across every agent in the chain is real.

The traffic you already pay for

Machines are not a future cost. They are most of the traffic hitting your site today, and every one of those visits spends something real: the server work to answer it, the bandwidth to deliver it, the crawl budget you meant for the pages that earn their keep. The machines that tell you who they are are the cheap ones. They announce themselves, they behave, and you can plan for them. The expensive visitors are the ones that hide, wearing a copied browser identity so you cannot tell them from a customer, and you serve them the full heavy page every time. The same structure that makes your site readable to a machine is the one that makes it cheap to serve. Legible and cheap turn out to be the same property, seen from two sides.

The machine journey

MX readiness is not binary. Machines attempt five sequential stages when they interact with a site, and failing at any stage blocks everything that follows.

Stage	Name	What it requires
1	Discovery	The site can be found: sitemap, robots.txt, llms.txt
2	Citation	Content can be identified and attributed: type, author, date, subject
3	Compare	Like-for-like comparison across entities: typed prices, availability, specifications
4	Price	Transaction-ready pricing: currency, validity, terms
5	Confidence	Trust signals: provenance, governance, permissions

Most sites pass Stage 1 and fail Stage 2. Everything from Stage 3 onwards depends on Stage 2 working. An AI assistant cannot compare your products if it cannot first cite what those products are.

Stage 4 hides a quieter failure. A page can be found, cited, and compared, and still mislead, because the date or the price on it lapsed months ago and the page never said so. An agent reads a timetable that still claims the train runs, or an open day that still says register now, and passes the stale fact on with full confidence. The fact was published. What was missing was any signal that it had expired, and to a machine a confident wrong answer is worse than none.

One more thing about that journey: only humans take journeys. People need guiding, step by step, because nobody absorbs a whole site at once. Machines are the opposite. Humans want a drip feed; machines, all of it at once. A machine lands on one page, reads what it can, and leaves. It might be a small model on a phone. It might be a scraper. It might be an assistant that only ever saw a text copy of your page, with the design stripped out. You cannot know which one arrives, and none of them see your layout.

The answer is redundancy. Carry the same meaning in your page structure, in your metadata, and in your plain text, so every kind of reader leaves with the same facts. Each page stands alone for the machine while the journey still works for the human. These are complementary designs on the same pages, not a trade-off.

What MX-ready looks like

A company that implements MX reaches a measurable destination. Agents can find your content, identify it accurately, and cite it without inference. Support queries that stem from agent misinformation decrease - fewer wrong answers means fewer tickets. Sales cycles that depend on machine-mediated comparison become faster, because agents can compare and commit without a human translating the page for them. Strategic assets (product knowledge, reviews, operational procedures) are sovereign and portable across any platform. You control what automated systems do with your content: you write the instructions, they follow them. No inference. No guessing. No dependence on AI getting smarter.

Consider what portability means in practice. If you have spent five years building reviews on a third-party platform and then migrate, every one of those endorsements stays behind. Zero reputation in the new system, despite years of earned trust. MX's Entity Asset Layer (covered fully in both books) changes this: your customer relationships, product knowledge, and brand signals become yours to carry, readable by any system, held by no single platform.

The window for first-mover advantage is measured in quarters, not years.

From the page to the file

The discipline this book describes is not really about web pages. It is about every file a company produces and depends on, and what happens to that file when it leaves the page it was first published on.

A contract is a file. A policy is a file. A product specification is a file. A regulatory submission is a file. A decision recorded by an AI system is a file. The same file might be served as a web page on Tuesday, attached to an email on Wednesday, sitting in a regulator's archive in three years, quoted by an agent in a customer-support conversation next month, and lifted into a court bundle in a dispute the year after. The bytes do not move. The context that made the bytes meaningful is what tends to fall away each time the file is read somewhere new. The work the cog format does is make that context travel with the bytes.

Two fields do most of that work. The `expires` field declares the date past which the publisher will no longer stand behind the content; a returns policy with an expiry tells any reader the file is canonical for a year and stale after. A web page has no equivalent. It is current until it stops being current, and there is no way to know which side of that line you are on by reading the page. The `canonicalUri` field declares the file's identity as distinct from its location. URLs are addresses; identity plus a registry is what survives a publisher's website being rebuilt twice.

The same instinct governs the links inside a file. A reference is only useful if the reader can resolve it from where they are. A link at the wrong depth, or one that assumes a folder the machine never established, points at nothing and says nothing about it. So MX writes references to resolve from the reader's actual position: the correct relative path inside a file tree, where any reader who has the file can follow it, and the full canonical address on the web, where the link must resolve with no file tree at all.

The convergence carries on into print. Most of the files that matter most in regulated life are still printed: solicitors' letters, prescription leaflets, planning notices, drug labels, loan agreements, building certificates, receipts. A QR code on the artefact encodes the file's identity. A machine reading the QR resolves to the canonical attested version and checks that the printed page matches what the publisher signed. The human reader still sees the printed page; the machine reader sees the cog; both see the same content; the machine sees it with full provenance, without optical character recognition and without a guess.

The fields that record this provenance travel across carriers without changing shape. A PDF carries them in its XMP metadata packet. An HTML page carries them in `<meta>` tags in the head. A markdown file carries them in YAML frontmatter. A JavaScript or shell script carries them in commented header blocks. The values are the same; the wrappers differ. JSON is the canonical form the standard publishes, because JSON is what every language can parse, and every other carrier is a transcription of the same JSON shape into the format that file already uses. So the same provenance question, asked of a PDF or a web page or a contract or a script, gets a structured answer drawn from the same dictionary, in whichever carrier the reader happens to hold.

The last carrier is the one you see. A card in a listing, a badge beside a title, a panel that reports what was checked: each is a component. The MX Component Standard asks it to bring the same fields to you and to the machine reading the page, mirrored into the markup, with the trust signals reported one by one rather than as a score.

> Drop the file on the inspector. The bytes never leave your machine. The tagged structure, the metadata packet, and the chain that names the human accountable for the record become visible in the browser, in seconds, without the command-line tools a regulator's audit team would otherwise reach for.
>
> One detection core grounds the public page anyone can run, the production gate every shipped file passes through, the test harness that catches regressions before they reach the public page, and the operator tool any procurement reviewer can install on their own machine. The mechanism is the credibility, seen from four sides.

The free shorthand: MX is for any file that needs to remain meaningful once it has left the page, the domain, the platform, or the format it was first published in. *MX: The Handbook* and *MX: The Protocols* pick up the file-level work in full. The point this introduction is making is that the scope is broader than your website, and the cost of pretending otherwise is the same silent loss as the web-only failures above, scaled across every file your company depends on.

When publish means public

Here is a failure that runs the other way. You paste a document into an AI assistant, click a button marked "publish", and copy the link it hands back. To you the word means what it means in a shared drive: an unlisted address, sent to the people you choose. To the machine web it means what it has always meant, a public page at a fixed URL that any crawler may read, index, and keep. On the consumer tiers of at least one major assistant, published documents are open to search indexing, and people have found contracts, salary calculators, and email lists sitting in a search engine's results. Taking the page down does not recall the copies already made, and on a domain you do not own you have no lever to force it. Treat every "publish" as public and permanent, and you will not be the one who finds out the hard way.

The trust layer: REGINALD

Stage 5 of the machine journey - Confidence - sits apart from the others. Stages 1 through 4 are about machine-readability: can a machine find your content, identify it, compare it, and complete a transaction? Stage 5 asks a different question: can a machine *trust* what it found?

That question is becoming structural. As AI-generated content multiplies, a machine reading your contract, your product specification, or your policy document faces a provenance problem that better markup cannot solve. Who wrote this? When was it last updated? Is the version it is reading current, or superseded? Was it written by a human, an AI, or an automated system?

This is not a web problem. It is a document problem. Every company publishes documents alongside its web presence - technical references, compliance filings, standard operating procedures, research reports. Most carry no machine-readable provenance. An agent reading them guesses. Guessing is how a three-year-old policy gets cited as current. Guessing is how

AI-generated content passes as editorial. Guessing is how trust erodes at scale, silently, without a complaint or a log entry.

REGINALD addresses this. The name is a traditional proper name that describes exactly what it does: Registry for Genuine Information, Notarised Authentication, and Legitimate Documentation. It is a public registry - the provenance layer for documents of any kind. When a document is registered, REGINALD records a cryptographic signature: who published it, that it has not been modified since publication, and when the current version was issued. The claim is narrow and precise: *this is what the owner published, unaltered.* REGINALD does not attest factual accuracy. It attests origin and integrity.

The same trust question applies to the tools you install, not only the documents you read. An editor extension or a package runs real code while telling you almost nothing about what it touches, and a marketplace listing is a promise nobody checks. The answer is the same three moves: the tool declares what it touches, a check confirms the declaration against its own code, and the publisher is bound by attestation, so a tool wearing a trusted name it has no right to is caught before it runs. It is the difference between a store label a developer types in and a claim that has been verified against the thing it describes.

The scope extends to any document any machine might encounter. A markdown specification, a PDF, a policy file on a corporate server, a technical reference in a git repository. The cog format - a record in a data file, conforming to a published standard, carried alongside the document it describes - is the unit. REGINALD is where cogs become discoverable and verifiable by any machine on earth, not just machines that already know where to look.

The practical effects compound. An agent that reads a provenance-attested document hallucinates less - it has verified origin and version to cite rather than gaps to fill by inference. Each task then needs fewer inference steps, so its token count and energy draw both drop. A chain of agents reading attested documents propagates verified provenance rather than re-deriving it at each step.

That saving is counted per task, and the total behaves differently. In 1865 the economist William Stanley Jevons answered a popular hope of his day, that more efficient steam engines would make Britain's coal last longer. He argued the reverse: each gain in efficiency made coal worth burning for work that had not paid before, so total use rose. The pattern carries his name, the Jevons paradox, and AI companies invoke it openly. TypeSafe AI called its decision model Jev, after Jevons, because it expects every tenfold drop in the cost of intelligence to unlock orders of magnitude more uses. The price is falling fast: Epoch AI measures the cost of reaching a given level of AI performance dropping to about a thirteenth each year since 2023 (Emberson & Roodman, 2026). Expect your documents to be read more often, by more machines, as that price falls. What you control is what each read finds: the answer you declared, or a fresh guess.

The regulatory dimension is arriving in parallel. The EU AI Act, the European Accessibility Act, and digital-records legislation across multiple jurisdictions are placing documentation, logging, and verifiability obligations on the companies they cover: can you demonstrate that a document is what it claims to be, and that the content an AI acted on was genuine? MX and REGINALD do not grant compliance with any of these regulations - that remains a legal duty of the company. What they do is make the documentation the company must produce structured, machine-readable, tamper-evident, and verifiable on request. REGINALD's attestation, in particular - cryptographically verifiable, portable, and document-native - travels with the document and lets any reader confirm that document's provenance at any point in the chain.

MX and REGINALD are the two pillars. MX makes content machine-readable. REGINALD makes it machine-trustworthy. And trustworthy is not one thing - it is a set of signals a machine can check one at a time. Who published this, and when? Is this the current version, or has a newer one quietly superseded it? Is the publisher still standing behind it, or have they gone quiet and left it to drift? REGINALD answers each of those as a separate, checkable signal, so a machine deciding on your behalf can see not just whether to trust a document but exactly what about it is missing. Stage 5 requires both pillars, and the trust pillar earns its name by answering those questions rather than asserting trust and hoping.

A careful buyer asks one more question. If a single company writes the rule and sells the verdict on whether you meet it, the verdict is worth nothing, because the seller can widen the gate whenever a sale depends on it. MX answers it by keeping the two apart. The definition of what counts as compliant is held in the open by The Gathering, the community body behind the standard; REGINALD is the commercial engine that measures against it. You can read the rule without paying, check your own work against it, or take the assessment elsewhere. The standard you build on is not a product you can be locked into.

> AI makes expert advice abundant. Peer wisdom is what it cannot manufacture. The Gathering exists to provide what no model can: the honest account of practitioners who have deployed the standard and will tell you what they found.

The padlock in a browser's address bar attests the pipe, not the page. It confirms that bytes arrived unaltered from a server; it does not say who wrote them, when, on what authority, or whether they have been changed since publication. A fraudster's site can carry exactly the same padlock as a bank. Human readers fill that gap with inference and mostly cope. Machine readers do not have the same instinct, which is why so many of them hallucinate. A signed cog closes the gap: the content itself can now attest the page, in a form a machine can actually check.

The same machinery that records who published a document also generalises. Where a document is the answer to "what does the company say?", the next category of record is the answer to "what did the company's AI system decide?". The same registry, the same cryptographic guarantees, the same separation of the record from the signature wrapping it. A regulator asking about an AI decision asks against the same fabric a customer asks about a refund policy. A decision model that answers in half a second for a fraction of a cent makes

those decisions plentiful, so the record has to keep pace: what the system was asked, how sure it was, the bar it was held to, and who confirmed the call when it fell short. The same goes for the regulatory codes themselves - a policy published as a record-in-a-data-file is addressable, versioned, and stable in a way a moving PDF is not. The trust-layer story does not stop at content. It extends to every artefact a regulator, an auditor, or a downstream agent might need to verify, on the same substrate, with the same primitives.

That alignment has practical consequences. UNESCO's Recommendation arrives with two assessment instruments: an Ethical Impact Assessment (EIA) and a Readiness Assessment Methodology (RAM). Both require evidence - declared values, measured outcomes, auditable processes - and neither has anywhere to land until the systems they assess have a technical substrate to assess against. MX is one such substrate. A document with semantic structure, declared metadata, accessibility tagging, and REGINALD-attested provenance gives an assessor concrete signals to score, rather than reasoning by inference. MX is not an ethics framework. It is the technical discipline that makes ethical assessment verifiable.

How regulation forces transparency

Platforms will not voluntarily solve the transparency problem. This is not cynicism. It is how incentives work. A platform profits from being the intermediary between reader and content. Solving the provenance problem means building the conditions for readers to verify content *without* using the platform's mediation. A platform will not teach you to make itself optional.

But regulators will. On 3 June 2026, the UK's Competition and Markets Authority issued Google a conduct requirement: show publishers exactly what their content does inside AI search features. Google had been silent on this. Publishers had no visibility into whether AI Overviews were driving traffic or stealing it. The zero-click rate was climbing - reaching 20% to 50% depending on category - and publishers were watching their traffic disappear into silence.

Google resisted the requirement. The CMA was unmoved. Google complied. Today, Search Console shows disaggregated AI search metrics: impressions, clicks, click-through rate, all per-URL. Publishers can see what is happening to their traffic in AI search. They can also control it: withhold content from AI answers if they choose.

This is transparency arriving through regulation, not goodwill. And it is the clearest proof yet of the larger pattern: platforms solve transparency problems when regulators make them, not when it is the right thing to do. The tracking Google provides is real and useful. It is also platform-controlled data, measured by metrics Google defines, on a schedule Google maintains, revocable at Google's discretion.

This is why you need both layers. Use the platform-provided tracking - it is free data from a regulator-forced win. Then build the layer you own. An RSS feed on your domain that tells machines what you publish. An llms.txt that names what machines may do with your content. Provenance metadata that travels with your documents, independent of which platform is

dominant this quarter. The platform's transparency is a window into one system. Your owned layer is the proof that survives all systems, because it does not depend on any of them.

Regulation is arriving across the board. The EU AI Act, the European Accessibility Act, and digital records rules expect companies to produce documentation that is structured, auditable, and verifiable on request. Waiting for platforms to teach you how to do that is waiting for help that will not come. Build the layer you own. The companies that do will be the ones that survive the next regulator's order, because they will already have what the order is demanding.

Why you need an audit, and why one is not enough

You cannot fix what you cannot see, and almost none of this is visible from where you sit. You read your website in a browser that runs the scripts, waits for the images, and hides the markup; a machine reads something else entirely, and the gap between the two is invisible to you until someone measures it. The same blindness runs deeper than the website. So the honest first step is not a redesign. It is to find out where you actually stand, and that takes three separate looks, because the failure hides in three separate places.

The first is the **site audit**. It answers one question: what does a machine actually see when it reads your content, and where is it forced to guess? Not what your page looks like to you, but what an agent extracts from it: which facts it can read, which it cannot, where the structure fails, where a price or a policy or a claim goes missing, where the machine fills the gap with a plausible invention instead. The outcome you buy is sight. You stop guessing what the machine makes of you and get a page-by-page account of what it reads, what it misreads, and what it cannot find at all, ranked by what it costs you when an agent gets it wrong.

The second is the **repository and CMS audit**, and it asks a question most teams have never thought to ask: can the place where you make your content be trusted to make it? Every major content system shipped AI write-access this year, and none of them shipped AI accountability. An agent can now edit your pages, and nothing records whether the agent or a person made a change, whether the change was reviewed, or whether the guard-rails that should have caught a bad edit were even switched on. The outcome you buy is trust in the source. You learn whether the system producing your content carries the provenance, the checks, and the accountability that let you stand behind what it publishes, or whether it is quietly letting machines write into your name with no record that they did.

The third is the **operator audit**, and it is the one nobody sells and everybody needs, because it looks at the people. Do your team actually know the tools they are running? Are they repeating the same AI mistakes week after week, losing hours to failures nobody has diagnosed? Have they, under deadline, quietly switched off the very guard-rails the other two audits put in place? A governed system decays the first Friday someone adds a bypass to get a release out, and no amount of good architecture survives an operator who does not know it is there. The outcome you buy is confidence that your people can run what you built: a clear read on where their time is lost, which mistakes recur, which tools around them are actually a risk, and where the discipline has already slipped.

The reason you need all three is that each one hides the others. A perfect site poured out of an untrustworthy system is a lie told fluently. A trustworthy system run by a team that has turned off its checks is a locked door with the key left in it. And a disciplined team publishing content a machine cannot read is effort spent talking to no one. The three audits are one practice, and they are a ladder: the site audit shows you what the machine sees, the repository audit shows you whether the place that made it can be trusted, and the operator audit shows you whether your people can keep it that way. Each finding becomes the syllabus for fixing the next, and each dated audit is the evidence of an accountable review that a regulator, a partner, or an acquirer will one day ask you to produce. Knowing where you stand is not the redesign. It is the thing that tells you which redesign is worth paying for.

Where to go from here

I first understood this at CMS Kickoff 2024 in Florida. A speaker asked the room how an eight-year-old buys a toy plesiosaur. The child searches, points, wants. They do not navigate carousels, read brand copy, or create accounts. Machines behave identically, at billions of times the scale. That observation turned my understanding of AI upside down. AI is better suited for *consuming* content than creating it. The websites need improving, not the AI.

That insight became two books.

MX: The Handbook

Practical patterns, working code, and implementation guidance for developers, content teams, and the companies that deploy them. Covers machine-readability (MX) and the document trust layer (REGINALD) in full.

#	Chapter	What it covers
1	Don’t Make AI Think	The worst-machine principle: why simpler HTML wins AI recommendations
2	How AI Reads	How agents fetch HTML, build a DOM tree, and extract meaning from structure
3	Guiding Principles	The core commitments: semantic clarity, structure, explicit metadata, redundancy, and marking what is material
4	Content Architecture for Machines	Articles, products, FAQs, navigation, and why retrofitting always costs more
5	Metadata That Works	JSON-LD, Schema.org, Open Graph, and YAML frontmatter in practice
6	Navigation and Discovery	How machines find content before they can read it
7	The JavaScript Challenge	The rendering divide and its direct commercial consequences
8	Testing with Machines	Six MX readiness levels with a complete testing methodology
9	Common Anti-Patterns	Fourteen recurring mistakes, each with before-and-after code
10	Implementation Roadmap	A phased approach that delivers measurable ROI at each stage
11	Business Imperative	The chapter to share upward: evidence to justify the investment
12	Cogs and REGINALD	REGINALD as the provenance layer: every file as a cog, the public registry that attests origin and integrity

MX: The Protocols

The technical reference for architects and practitioners who need to understand the full scope of what machine experience means, and what to do about it. Covers both pillars: machine-readability (MX) and machine-trust (REGINALD).

#	Chapter	What it covers
1	What You Will Learn	Orientation and scope
2	The Invisible Failure	Silent breakdowns: invisible form failures, vanishing errors, pixel-only state
3	The Architectural Conflict	Why modern web design is structurally hostile to machines, by design
4	Business Reality	The commercial case: what machine failure costs, and what readiness returns
5	The Content Creator's Dilemma	Machine-mediated discovery and advertising-funded publishing
6	The Security Maze	Machines inheriting authenticated sessions, acting on behalf of users
7	The Legal Landscape	Jurisdictions writing rules for a problem that has already scaled
8	The Human Cost	The digital divide and the machine divide: same causes, same fix
9	The Platform Race	Amazon, Microsoft, Google, Anthropic: machine commerce in seven days
10	Generative Engine Optimization	Discovery patterns that work across SEO and GEO simultaneously
11	Designing for Both	Serving machines and humans from shared structural principles
12	Technical Advice	Working code, testing strategies, and implementation tools
13	What Agent Creators Must Build	The developer side of the machine relationship
14	Agent Protocols	Standards and patterns for agents visiting your site
15	Intent-Driven Publishing	Metadata-first architecture where structure drives content assembly

#	Chapter	What it covers
16	When Machines Remember	Sovereign, portable machine memory built on open standards
17	The Joymaker	A 1969 science fiction novel that predicted machine-readable architecture
18	Content That Manages Itself	Five generations of CMS, and why the current model is transitional
19	The Information Architecture of the LLM Web	A Three-Layer IA Framework synthesising everything
20	Cogs and REGINALD	How REGINALD establishes provenance and trust: the cog format, attestation, and the document registry for any machine on earth
21	The Fields and the Standards	Protocol proposals that give the architecture durability
22	Content Negotiation and the Markdown Trap	The format layer that determines what machines can access

---

MX for your own repository

The Protocols turns the whole idea inward in a later chapter, and it is the one to read if you build or maintain software. Everything this introduction said about a web page, that it is a publishing point and must declare what it is to the machine that reads it next, is just as true of the repository where your work is built. A repository hands files to machines all day: to the AI agent editing a branch, to the build that runs on every change, to the new colleague on their first morning, to another project that depends on yours. Point Machine Experience at that repository and it begins to describe itself, check its own rules, and repair its own drift, so the next thing that opens it, human or agent, walks into a place that already knows what it is. Give it that discipline and the repository becomes a governed execution surface: the place an agent acts on your work safely and on the record, within bounds a human sets and against checks that refuse whatever does not conform. The chapter also names the rule that keeps a self-mending repository from quietly harming itself: it should only repair itself when it can see its whole self, because a repository that heals from a partial view heals wrong, and the damage looks exactly like tidying. The full treatment, the three properties and the boundary that makes them safe, is in The Protocols. If your estate includes a codebase, and it does, that chapter is the reason to buy the book.

MX: The Handbook is available now and MX: The Protocols ships July 2026; reserve your copy and find both at https://mx.allabout.network/books/. They share continuously updated appendices at https://mx.allabout.network/books/appendices/. The Protocols is where the repository chapter, and the full scope of both pillars, lives in detail.

The standard itself is not mine to keep. The Gathering - the open, vendor-neutral standards body that governs MX - makes the rules and the enforcement of the rules visible to everyone who depends on them. That is UNESCO's fifth principle applied in practice: responsibility and accountability require governance any stakeholder can verify, not vendor decisions held in private. The Gathering's founding sponsors fund the work without owning it; the standard belongs to the community.

> The Gathering owns the open standard. CogNovaMX owns the operating rules around its commercial offerings. Founding sponsors declare a focus within the standard's scope and choose at their discretion whether to engage with the operating layer.

CogNovaMX delivers the audit as a partner service, as a licensed tool that runs on your own infrastructure, or as a white-label arrangement under your brand. For companies that cannot share infrastructure - regulated industries, local businesses requiring data sovereignty - the tool runs on a NAS device on your own premises or on a hosted instance reserved exclusively for you. In each case the assessment is independent: unlike a platform vendor's own diagnostics, the audit serves no algorithm of its own. That independence is the point. The audit identifies the platform and content management system behind a site and benchmarks it against peers on the same stack, so it reads from the machine's side whether the publishing point you depend on is keeping the contract described earlier in this chapter, and tells you whether the gap is yours to close or your vendor's.

The structural failures described in this chapter do not fix themselves. They require deliberate work, and that work takes time. Starting before the wave breaks is not premature tuning. It is the minimum responsible position.

Enjoyed the Free Book?

Continue reading with the full books.

Inside the Books

A small AI model, asked who is accountable for a set of documents, named an owner for nine files in ten that named nobody. Given the same files with MX metadata embedded in them, it got every one right, in about a tenth of a second and at no cost.

MX: The Protocols shows why declared metadata is worth most to the smallest, cheapest models, and why cheap machine decisions multiply guesses unless the documents declare their state. MX: The Handbook shows why a fashionable tactic such as GEO is a safe purchase rather than a sound one, and what lasts after the fashion moves on.

MX: The Protocols - The complete technical reference: every pattern, every principle, every implementation detail for Machine Experience.

MX: The Handbook - The practical guide: audits, checklists, and step-by-step instructions to make your content work for every machine.

Purchase the books at: https://mx.allabout.network/books/index.html

Originally published at
https://cmscritic.com/a-cms-consultants-takeaways-from-cms-kickoff-2024

The AI Tipping Point: A Consultant's Takeaways from CMS Kickoff 2024

Tom Cranstoun · Principal Consultant, Digital Domain Technologies Limited

---

As a first-time attendee at the Boye & Company event, I didn't know what to expect. But the speakers and content left me with a deeper understanding of the future of CMS - and how AI is changing everything.

Tom Cranstoun is the principal consultant at Digital Domain Technologies Limited and a CMS Critic contributor.

Reflecting on the Boye & Co CMS Kickoff 2024 down in St. Pete Beach, it's clear we're at a tipping point with AI in the CMS space. The event was a melting pot of CMS enthusiasts and experts, all there to predict and shape how we handle CMS. The setup was well-suited for learning, networking, and bouncing ideas off one another, with sessions covering a wide range of topics.

We had some standout talks, like Deane Barker's Q&A and Kristina Podnar's insights into Generative AI. Becky Brown and David Rasmussen hit home the importance of change management for content teams. Mike Wills' take on flexible content models for multichannel strategies was enlightening, and Jeff Eaton shed light on how CMS page builders affect team dynamics.

Brian Browning showed us practical AI applications that are changing digital experiences. Debbie Tucek pushed us to think of content as a product, while Ricky Frohnerath's masterclass on high-performance content teams was a game-changer. Nick Rudd's perspective on AI in real-time content generation was eye-opening, and sessions by Marli Mesibov and Brian McKeiver on the CMS buy vs. build dilemma and digital commerce were thought-provoking.

The twist in my expectations

I noted that the agenda of the CMS Kickoff 2024 offered only a *slight* focus on AI. The speakers often mentioned it during their talks; it's very "of the moment" in our industry.

I, like you, have been to many seminars, presentations, and conferences around AI and what it will do for you and your content. I was pleasantly surprised this was not about AI *creating* the content - it was about AI *reading* the content. It turned my expectations upside down, fired up my brain, and I could not stop taking notes.

The whole CMS Kickoff conference was not just about AI. I have many takeaways that I am working on. But the most important is this: *the AI revolution will change our industry and everyone else's*. Still, despite its great promise and clear benefits, it soon becomes clear that using AI to create content is not ideal. In fact, AI-generated content has many problems, including:

- Enforcing the use of internal style guides.
- Maintaining a brand tone of voice.
- Avoiding gender or race biases.
- Avoiding cultural nuances.
- Knowing which sources were used in the training of the AI (too few sources means limited vocabulary - conversely, too many sources suggest the corporation loses control. Does the AI manage to use local idiomatic expressions? For instance, a "sidewalk" is a "pavement" in the UK).
- AI sometimes gives up asking the prompter to continue.
- AI is becoming regulated, and legal ramifications will soon become commonplace, changing in each jurisdiction.

And this is just the beginning.

Rethinking AI's role in content management

Throughout the event, the speakers made me rethink AI's role in content management. Given the translation, bias, regulation challenges, and lack of trust in AI, one needs to employ reviewers and editors to correct the content - because AI is better suited for *consuming* content.

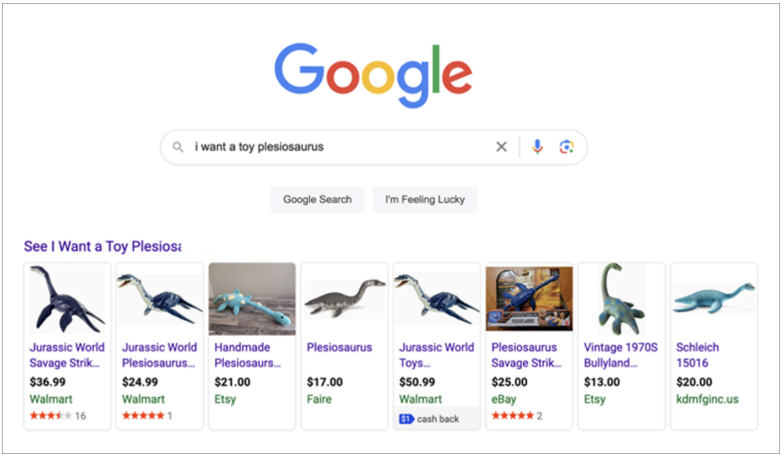
Let's think that the current state of AI creating content is a beta experiment, best left to the tech guys. After all, we have a business to think of.

Nick Rudd led an engaging discussion explaining how AI should be thought of. He began by asking how an eight-year-old purchases toys. He described it as simple and direct, without the baggage of brand loyalty or complex decision-making.

Kids are smart. They know what they want.

Ignoring the elephant in the room, "TikTok" (an entirely different article), the eight-year-old goes to Google/Bing/Whatever and types in:

"I WANT A TOY PLESIOSAURS"



Search results for “I WANT A TOY PLESIOSAURS”

What didn’t the eight-year-old do?

- They didn’t go to Amazon.
- They didn’t refine the search.
- They didn’t look at brands.
- They didn’t read customer reviews.
- They didn’t sort by price or popularity or whatever.
- They didn’t read the price (well… mostly).
- They didn’t create an account.
- They didn’t click on dropdowns.
- They ignored video and animations.
- They didn’t read the copy.

What happens when AI reads content? It behaves like an eight-year-old:

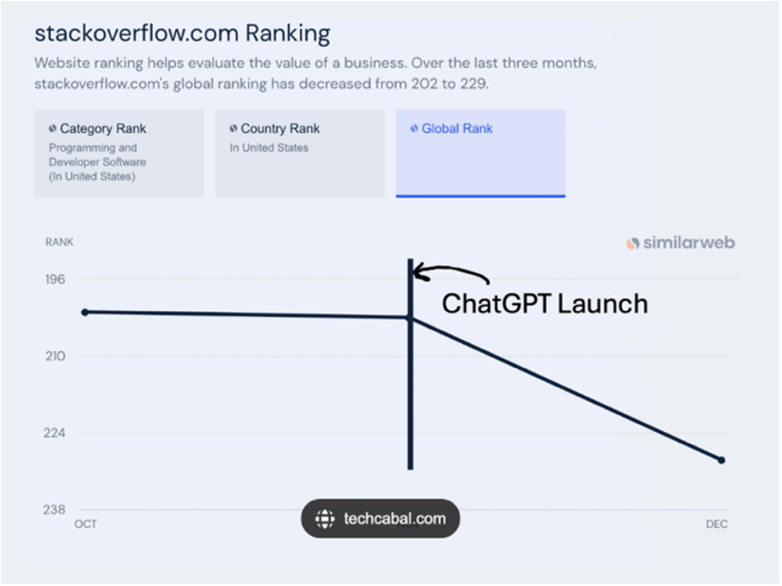
- It will ignore retailers.
- It will probably not refine the search.
- It will ignore brands.
- It might skim customer reviews.
- It might sort by price.
- It won’t create an account.
- It won’t follow your dropdowns.
- It won’t give you an email address (you no longer know *who* your customers are).
- It will ignore the videos and animations.
- It will ignore the “see our shop links”.
- It will skim-read the copy if it can find it among the ads.

How will “AI simplicity” affect your business?

The idea of simplicity will challenge how AI interacts with our current web designs and content structures.

The next step in making money from AI is that the AI offers to *purchase* the toy - keeping any discounts, commissions, and recommendation fees bypassing the retailer. This will impact KPIs as AI activity increases and human interaction decreases.

What happened to StackOverflow recently will begin to affect us all. The online Q&A platform for developers is releasing 28% of its staff to save costs, which is not necessarily related to AI. But look at the point where ChatGPT entered the picture:



StackOverflow traffic graph showing decline after ChatGPT launch

What about a site’s KPIs?

As AI interaction increases, our reliance on traditional KPIs must fall.

Every website today uses analytics to drive statistics, such as:

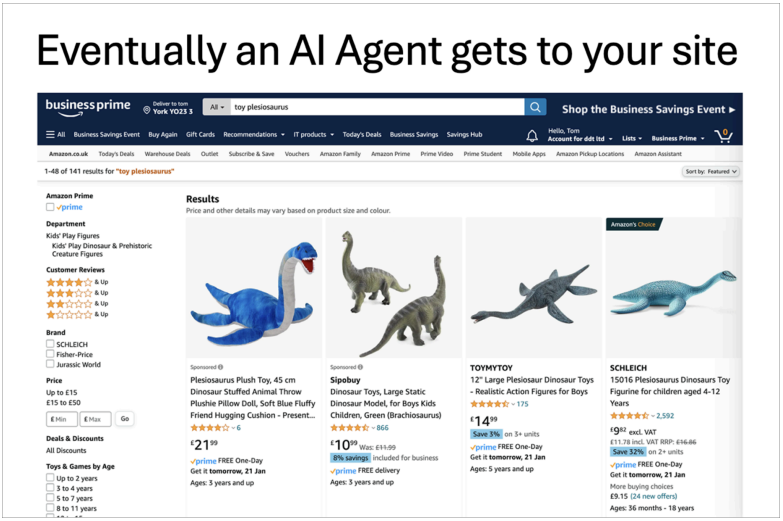
- The number of total visits.
- The number of unique visitors.
- The number of new vs returning visitors.
- Geographical location of visitors.
- Net promoter score.

- Pages viewed per session.
- Dwell time.
- Bounce rate.
- Conversions, including A/B testing.
- Organic search visitors.
- The number of social share impressions.
- Feedback and reviews.
- Cart abandonment.

Design by engineers: a trap

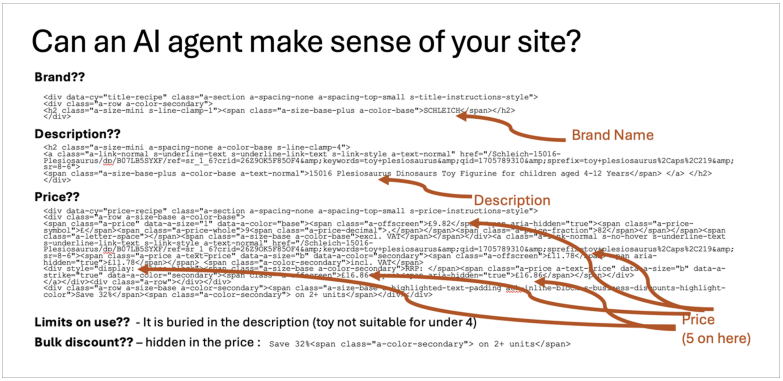
The conference also highlighted a common pitfall: websites designed by engineers focus on *looks* over *function*, which does not work well for AI. We need a more structured approach to the content model for better clarity and order.

Today's HTML markup makes sense to the browser:



Clean product page rendered in browser

However, opening the "View Page Source" shows a different take:



Messy HTML source code of the same product page

Buried in the HTML are the brand name, the price (more than one is shown in the HTML), and the description. Note that discounts and age suitability are placed in the text. A human reading this would have difficulty, never mind an AI.

Engineers create text and image boxes in your CMS; they let you fill and serve them.

They might call them hero's carousels, cards, etc. They are dressed up text and image boxes with interactions.

This is the *Design by Engineer Trap*.

What needs to be added to our sites?

The content model and schema presentation!

A lot has gone wrong. The IT people who made this website are very clever engineers; I could not do this.

The engineers took the site owners' desires and made pretty pictures with text and prices, highlighting discounts. They made it reasonably fast and effective and made the site owner one of the world's wealthiest men.

Look at other sites' HTML. How do they handle pricing? *Awful!*

Each site dresses up text in presentations. No semantic meaning is given to the objects described. There is no content model. It looks like AI will need an LLM for each brand!! This is not going to work in the AI world.

Something *major* must happen…

The content model, the traditional CMS way

You probably already know it: the model gets built into the CMS.

This is a trap.

Designers fixate on the presentation when envisioning a website - and build a style guide with fonts, palettes, components, and interactions. They create heroes, cards, lists, articles, carousels, accordions, tabs, text, and much, much more.

What next?

The content team is asked to create:

- **Testimonials.** *Someone decides that testimonials should be in the cards component.*
- **Special offers.** *Someone decides that testimonials should be in the lists component.*
- **A "coming soon" message.** *Someone decides that testimonials should be in the image with a link component.*
- **Bulk discount messages.** *Someone decides that Bulk Discount should be in the text component.*

The content model, built by developers!

So, what do we need to do about this?

We need technical improvements, enthusiastic use of content models, and application of schema without putting cognisant overload on the content team - which is a big ask.

I assure you it will be worth it.

Let's look at the HTML example we showed earlier, this time with schema applied:

```
<div itemscope itemtype="http://schema.org/Product">
<h1 itemprop="name">Plesiosaurus</h1>
<p itemprop="description">15016 Plesiosaurus Dinosaurs Toy Figurine.</p>
    <div itemprop="audience" itemscope itemtype="http://schema.org/PeopleAudience">
        <meta itemprop="suggestedMinAge" content="4">
        <meta itemprop="suggestedMaxAge" content="12">
        Suitable for ages 4-12.
    </div>
    <div itemprop="brand" itemscope itemtype="http://schema.org/Brand">
        <span itemprop="name">SCHLEICH</span>
    </div>
    <div itemprop="offers" itemscope itemtype="http://schema.org/Offer">
        <span itemprop="priceCurrency" content="GBP">Price: £</span>
        <span itemprop="price" content="11.98">11.98</span>
        <link itemprop="availability" href="http://schema.org/InStock" />In Stock
        <a itemprop="url" href="http://example.com/product">Product Page</a>
        <div itemprop="eligibleTransactionVolume" itemscope itemtype="http://schema.org/PriceSpecification">
        <meta itemprop="minValue" content="2">
        <span>Bulk discount for 2 or more units.</span>
    </div>
</div>
```

HTML source code with Schema.org markup applied

Now a machine can see the price, the description, the brand, the discount, and the age applicability. It knows it is for a product, not an ad on the site.

Call to action

We are in marketing, so we must end with a call to action. So here it is:

Our websites need to target four device types: *Mobile*, *Tablet*, *Desktop,* and *Machine* in the future. Our CMS strategy needs a solid content model that applies consistently across all platforms. This is key for targeted ads, tailored content, and campaign management.

However, the current developer-driven content-model approach must be revised and rethought. We must wrap our content semantically and stick to structured data schemas. Schema.org is the place to start; that is the technical part. We as an industry need to make the case and build a joint effort from designers, developers, and content creators to ensure content is built within the model and aligns with the established schema.

As such, teams need to add an "AI Evangelist" to the roles.

The AI Evangelist will be a key figure in our companies, driving the adoption and implementation of AI technologies. This role bridges technical knowledge, strategic insight, and the ability to communicate AI's benefits to diverse audiences. The AI Evangelist will forge partnerships across design, development, content, and testing teams to craft a unified AI vision.

Key Responsibilities of an AI Evangelist:

- Champion AI technology adoption, showcasing tangible improvements to digital strategies and user experiences.
- Lead AI-centric models and strategies, focusing on predictive analytics, tailored content, and AI-powered search tuning.
- Facilitate AI integration into workflows with cross-functional teams, ensuring smooth transitions and efficiency.
- Lead educational initiatives like workshops and training to build an AI-aware culture.
- Monitor and assess AI trends for strategic adoption.
- Cultivate a network with AI vendors and thought leaders to build expertise.
- Advise leadership on AI integration, securing necessary resources and investments.
- Examine new opportunities to decrease time-to-market and increase engagement.

Qualifications:

- Proficient in digital marketing, SEO, development, and UX design.
- Strong communicator, adept at explaining complex technology to varied audiences.
- Proven leader and collaborator in team-driven environments.
- A commitment to learning and staying current with AI developments.

Desired Skills:

- Background in digital analytics and tailored content strategies.
- Knowledge of CMS and digital content creation.
- Experience with app-based content platforms and audience engagement.
- Strategic thinker, ready to foster innovation and change.

Closing

CMS Kickoff 2024 was a significant event for me.

As an industry, we must reconsider our content strategies in the face of AI's evolution. As we gear up for AI's continued integration into our digital spaces, adopting a structured, semantic approach to content management is more important than ever in building cross-team strategies.

These reflections aren't set in stone but are a "call to arms" for the industry. Together, we must prepare for a significant shift in how we design, manage, and deliver content in an AI-first world.

Work with the Author

Tom Cranstoun has spent 25 years helping companies get their content right: first for humans, then for search engines, and now for every machine that reads content. As Principal Consultant at CogNovaMX, he brings MX from theory to practice.

Consulting

A structured engagement to assess your current web estate and build an MX roadmap. We audit your pages the way agents see them: toast notifications, sliders, stepped workflows, hidden JavaScript variables, and every interaction pattern that breaks for machines, not just metadata and markup. We identify where revenue is leaking to competitors and deliver a prioritised implementation plan your team can execute immediately.

Training

A one-day hands-on workshop for your development, content, and product teams. Participants leave with practical skills: semantic HTML patterns, Schema.org implementation, agent testing techniques, fixing interaction patterns that fail for machines, and a working MX template for their own site. Available on-site or remote.

Speaking

Conference keynotes, lightning talks, and executive briefings on Machine Experience: what it is, why it matters commercially, and what companies should do now. Tom is a regular speaker at CMS Experts events in Frankfurt, Florida, and London. The concept behind MX crystallised at a CMS Experts event in Florida - the plesiosaur moment that opens this book.

Ready to make your web work for every machine on earth?

Email	info@cognovamx.com
Web	https://allabout.network
Books	*MX: The Handbook · MX: The Protocols*

Get in touch to discuss how MX can work for your company.

---

*This book was published by **MX Printworks** - the publishing arm of CogNovaMX.*

Need a book that machines can read as well as humans? MX Printworks produces books and publications built for the AI age. Every title ships with semantic structure, machine-readable metadata, and companion web assets. From manuscript to print-ready PDF, we handle the entire pipeline.

MX Printworks via mx-printworks@cognovamx.com · https://mx.allabout.network/about/printworks.html



Photograph by Marta Cukierman - linkedin.com/in/cukiermanm/

CogNovaMX

ISBN 978-1-0676384-1-2

The web is no longer consumed only by people.

AI agents are becoming a new consumption layer for enterprise digital platforms — interpreting, transforming, comparing, and acting on information on behalf of users, teams, and systems.

That changes what "quality" means.

Not just user experience.

Machine Experience (MX).

MX is the discipline of designing digital systems so that machines can read, trust, and act — reliably, safely, and consistently — across channels, devices, and emerging agent platforms.

This advance preview introduces the foundations of MX: how agent mediation alters discovery, governance, compliance, and operational load — and why today's web patterns do not hold when the primary consumer is no longer a browser.

Tom Cranstoun has shaped the technology industry for over 40 years, building products and systems used by millions. A long-standing member of the CMS Experts community, he has worked with organisations including Nissan, Ford, Jaguar Land Rover, and Twitter/X.

In 2024, his CMS Critic article identifying the "AI tipping point" reframed the conversation: designing for machines is now as important as designing for humans.

MX: The Handbook develops the full framework — the language, principles, and practical patterns for teams building the next generation of enterprise digital platforms.

Full release: MX: The Handbook
allabout.network/mx

This preview is the beginning.

MX: The Handbook provides the full framework — practical patterns, technical standards, and implementation guidance for anyone building or managing digital experiences in the agentic era.

If machines are reading your website, this book will change how you build it.

PUBLISHED AND PRINTED BY MXPRINTWORKS.COM 120 MAIN STREET, LARGS, SCOTLAND © CogNovaMX. First published 2026. All rights reserved.